Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра О7 «Информационные системы и программная инженерия»

Практическая работа №5
по дисциплине «Информатика: Основы программирования»
на тему «Функции»

вариант 12

Выполнил:
Студент Праудин Д.С.
Группа 0717

Преподаватель:
Палехова О. А.

Санкт-Петербург
2021 г.

<u>Задание 1.</u> В матрице A (m,n), состоящей из нулей и единиц, найти квадрат заданного размера (квадратную подматрицу), состоящий целиком из нулей.

Исходные данные:

Результирующие данные:

Таблица тестирования:

Входные данные	**Ожидаемый результат**	**Результат работы программы**

Основной алгоритм:

Вспомогательные алгоритмы:

1 Алгоритм ввода массива

Входные данные: имя массива, размер.

Результирующие данные: заполненный массив

Схема алгоритма:

Этот алгоритм описывается в программе функцией

Параметры:

первый параметр

второй параметр

Возвращаемое значение –

Вспомогательные переменные:

2 Алгоритм вывода массива

Входные данные: имя массива, размер.

Результирующие данные: нет

Схема алгоритма:

Этот алгоритм описывается в программе функцией

Параметры:

первый параметр –

второй параметр

Возвращаемое значение отсутствует.

Вспомогательные переменные:

3 Алгоритм

Этот алгоритм описывается в программе функцией

Параметры:

первый параметр – количество строк

второй параметр – количество столбцов

Возвращаемое значение –

Вспомогательные переменные:

4 Алгоритм

Входные данные: имя массива, размер,

Результирующие данные:

Схема алгоритма:

Этот алгоритм описывается в программе функцией

Параметры:

первый параметр

второй параметр

Возвращаемое значение –.

Вспомогательные переменные:

5 Алгоритм

…

Текст программы

Задание 2.

Дана матрица D размера nxm (n,m □ 20). Сформировать три одномерных массива: A – из элементов матрицы D, лежащих в диапазоне от -5 до 5, в порядке следования по строкам, B – из элементов от 10 до 20, C – от 30 до 50. Использовать подпрограмму для выборки в новый массив элементов одного диапазона.

Исходные данные:

*x массив типа int (для записи чисел из матрицы в определенном диапазоне), матрица A[m][n] типа int (исходная матрица), переменная x1 типа int (перемещение по массиву x), переменная flag типа int(распределение по даипазонам)

Результирующие данные:

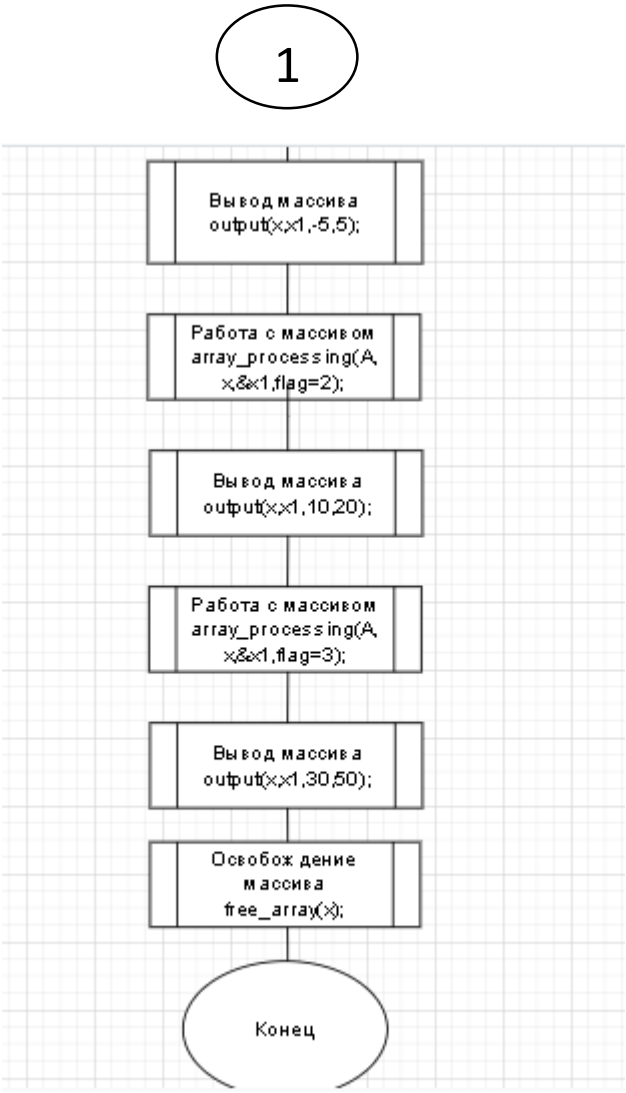
Массив x типа int (вывод массива в зависимости от диапазона)

Таблица тестирования:

Входные данные	**Ожидаемый результат**	**Результат работы программы**

1 2 3 4	Диапазон [-5;5]:1 2 3 4 Диапазон [10;20]: Диапазон [30;50]:	1 2 3 4 диапазон [-5, 5]:1 2 3 4 диапазон [10, 20]: диапазон [30, 50]:
3 12 35 55	Диапазон [-5;5]:3 Диапазон [10;20]:12 Диапазон [30;50]:35	3 12 35 55 диапазон [-5, 5]:3 диапазон [10, 20]:12 диапазон [30, 50]:35 Process returned 0 (0x0) execution time : 32.844 s Press any key to continue.
-7 23 56 200	Диапазон [-5;5]: Диапазон [10;20]: Диапазон [30;50]:	-7 23 56 200 диапазон [-5, 5]: диапазон [10, 20]: диапазон [30, 50]: Process returned 0 (0x0) execution time : 18.295 s Press any key to continue.
1 1 1 1	Диапазон [-5;5]:1 1 1 1 Диапазон [10;20]: Диапазон [30;50]:	1 1 1 1 диапазон [-5, 5]:1 1 1 1 диапазон [10, 20]: диапазон [30, 50]: Process returned 0 (0x0) execution time : 4.870 s Press any key to continue.

Основной алгоритм:



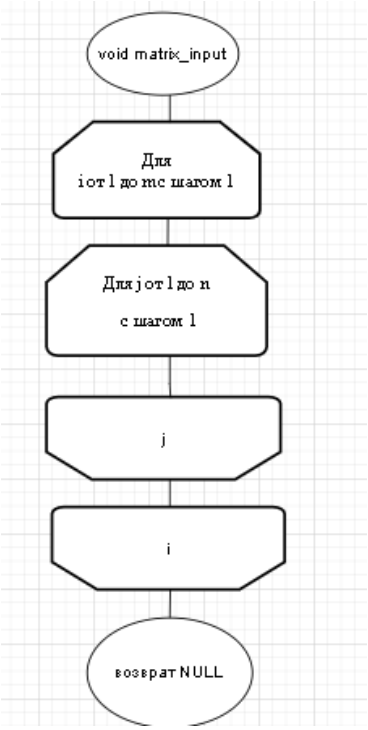
Вспомогательные алгоритмы:

1 Алгоритм ввода матрицы и описывающая его функция

Входные данные: Матрица A[m][n]

Результирующие данные: заполненная Матрица A[m][n]

Схема алгоритма:



Этот алгоритм описывается в программе функцией:

Прототип функции `void matrix_input(int A[m][n]);`

Параметры : int A[m][n] –пустая матрица

Возвращаемое значение –NULL

Вспомогательные переменные:

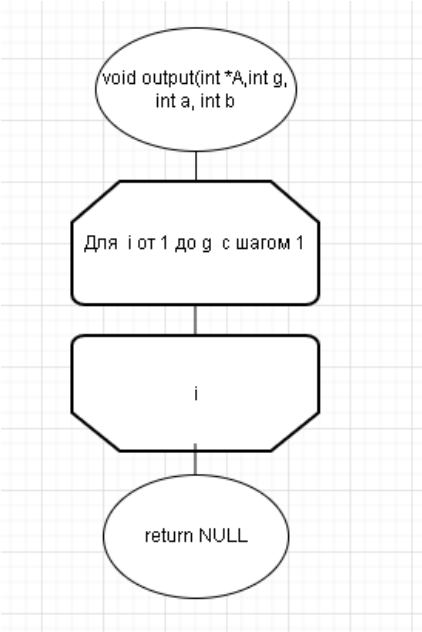
Переменные i и j типа int для перебора матрицы

2 Алгоритм вывода массива и описывающая его функция

Входные данные int *A,int g, int a, int b

Результирующие данные: массив A

Схема алгоритма:



Этот алгоритм описывается в программе функцией:

Прототип функции `void output(int *A,int g, int a, int b);`

Параметры `int *A,int g, int a, int b`

Возвращаемое значение –NULL

Вспомогательные переменные:

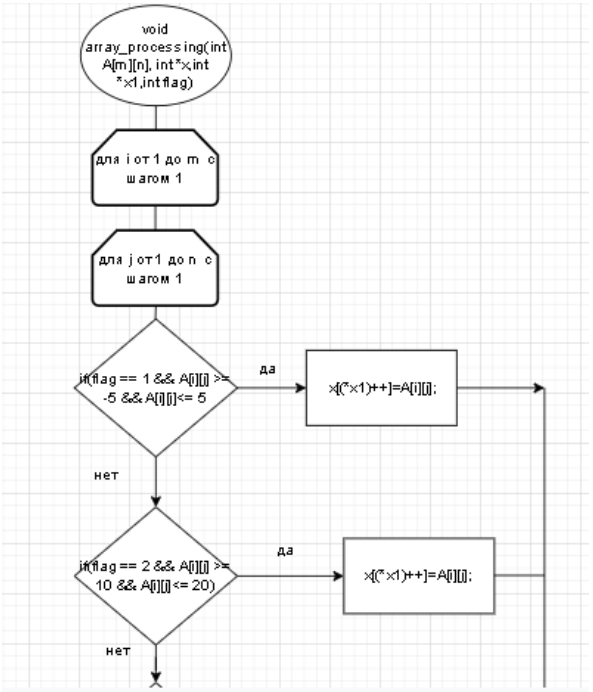
Переменные i типа int для перебора массива

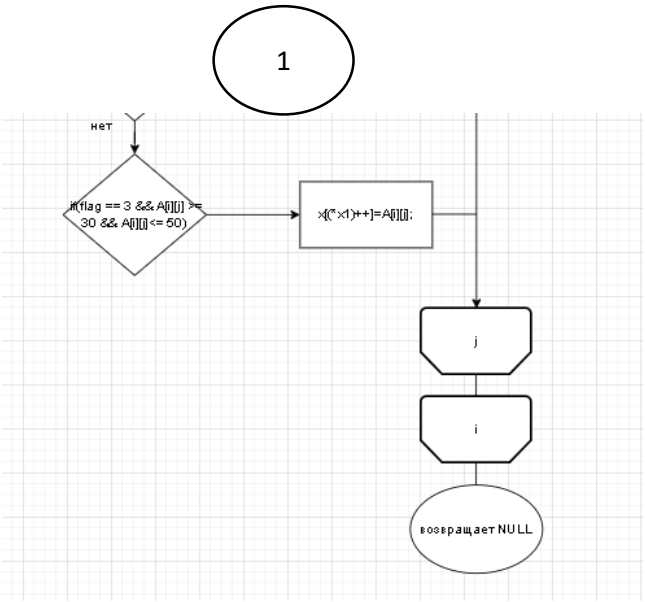
3 Алгоритм работы с матрицей и описывающая его функция

Входные данные int A[m][n], int *x,int *x1,int flag

Результирующие данные: Массив x

Схема алгоритма:





Этот алгоритм описывается в программе функцией:

Прототип функции `void array_processing(int A[m][n], int *x, int* x1,int flag);`

Параметры `int A[m][n], int *x, int* x1,int flag`

Возвращаемое значение –NULL

Вспомогательные переменные:

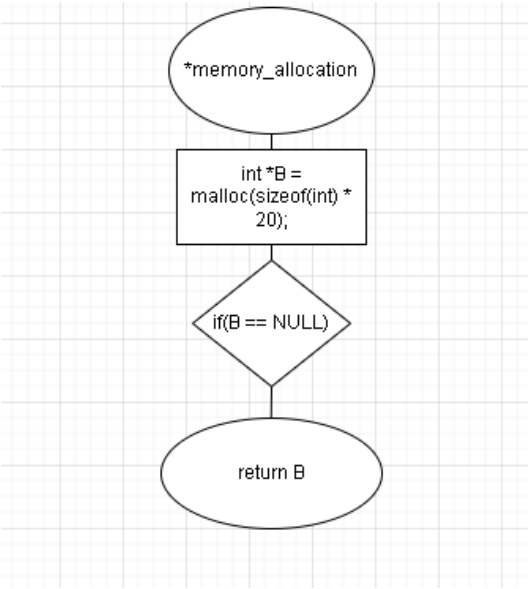
Переменные i и j типа int для перебора матрицы

4 Алгоритм выделения памяти под массив x и описывающая его функция

Входные данные : -

Результирующие данные: выделение памяти под массив x

Схема алгоритма:



Этот алгоритм описывается в программе функцией:

Прототип функции `int *memory_allocation();`

Параметры : –

Возвращаемое значение :B

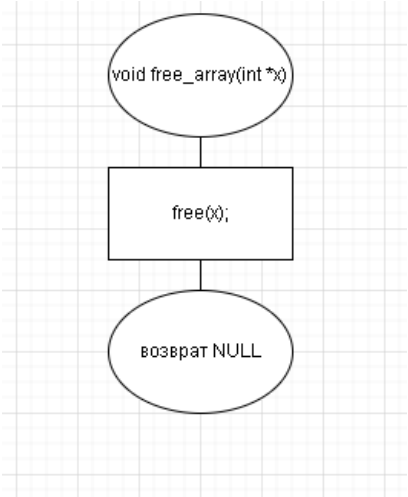
Вспомогательные переменные: -

5 Алгоритм освобождения памяти под массив x и описывающая его функция

Входные данные : int *x

Результирующие данные: освобождение памяти массива x

Схема алгоритма:



Этот алгоритм описывается в программе функцией:

Прототип функции `void free_array(int *x);`

Параметры : `*int x`

Возвращаемое значение - NULL

Вспомогательные переменные: -

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#define m 2
#define n 2
int *memory_allocation();
void array_processing(int A[m][n], int *x, int* x1,int flag);
void output(int *A,int g, int a, int b);
void matrix_input(int A[m][n]);
void free_array(int *x);
int main(){
setlocale(LC_ALL,"Rus");
int *x = memory_allocation();//выделение памяти под массив
int A[m][n]; //исходная матрица
matrix_input(A); //заполнение матрицы
int x1; //для перемещения по массиву x
int flag=0; //для разбивки по диапазонам
x1=0;
```

```
array_processing(A, x, &x1,flag=1); //обработка исходной матрицы
output(x,x1,-5,5); //выввод чисел в диапазоне
array_processing(A, x,&x1,flag=2); //обработка исходной матрицы
output(x,x1,10,20); //выввод чисел в диапазоне
array_processing(A, x,&x1,flag=3); //обработка исходной матрицы
output(x,x1,30,50); //выввод чисел в диапазоне
free_array(x); //освобождение памяти
return 0;
}
void matrix_input(int A[m][n]){ // функция  заполнения матрицы
int i,j;
for(i=0;i<m;i++)
for(j=0;j<n;j++)
scanf("%d", &A[i][j]);
}
void  array_processing(int  A[m][n],  int  *x,int  *x1,int  flag){  //функция
обработки матрицы, распределение по массивам
int i,j;
*x1=0;
for(i=0;i<m;i++)
for(j=0;j<n;j++){
if(flag==1&&A[i][j]>=-5&&A[i][j]<=5)
x[(*x1)++]=A[i][j];
if(flag==2&&A[i][j]>=10&&A[i][j]<=20)
x[(*x1)++]=A[i][j];
if(flag==3&&A[i][j]>=30&&A[i][j]<=50)
x[(*x1)++]=A[i][j];
}
}
void output(int *A,int g, int a, int b){ //функция вывода массивов
int i;
printf("диапазон [%d, %d]:",a,b);
for(i=0; i<g; i++)
printf("%d ",A[i]);
```

```
printf("\n");
}
int *memory_allocation(){ //функция выделения памяти
int *B = malloc(sizeof(int) * 20);
if(B == NULL) // проверка на то , что выделилась память
return 0;
return B;
}
void free_array(int *x){  //освобождение памяти
free(x);
}
```

Задание 3.

Определить функцию Integral() для приближенного вычисления определенного интеграла вида методом трапеций. Использовать эту функцию для вычисления значений двух интегралов, передавая подынтегральную функцию в функцию Integral() в качестве параметра.

Формула трапеции $\int_a^b f(x)\,dx \approx \frac{b-a}{N}\left(\frac{y_0+y_n}{2}+y_1+y_2+\ldots+y_{n-1}\right)$ – формула трапеций;

Исходные данные:

double f1(double x) и double f2(double x)

Результирующие данные:

значение интеграла – вещественное число

double temp1 double temp2

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
$\int_{-2}^{2}\sqrt{1+x+x^2}\,dx$	5.77	5.770312 1.813562 Process returned 0 (0x0) execution time : 0.123 s Press any key to continue.
$\int_{0.5}^{4} x\sin x\,dx$	1.81	5.770312 1.813562 Process returned 0 (0x0) execution time : 0.123 s Press any key to continue.

Схема функции Integral:

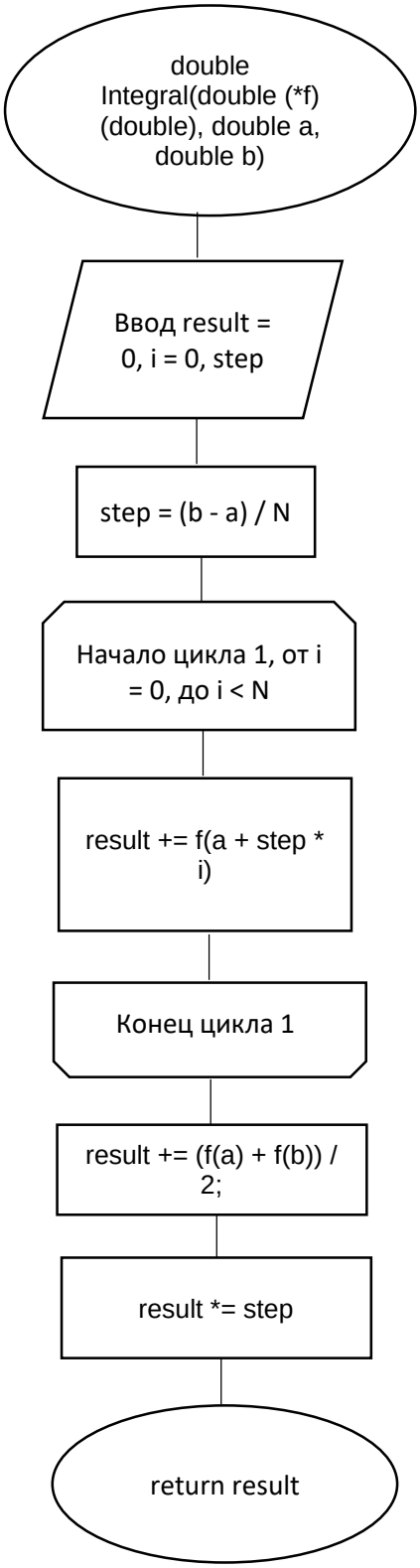


Схема функции main:

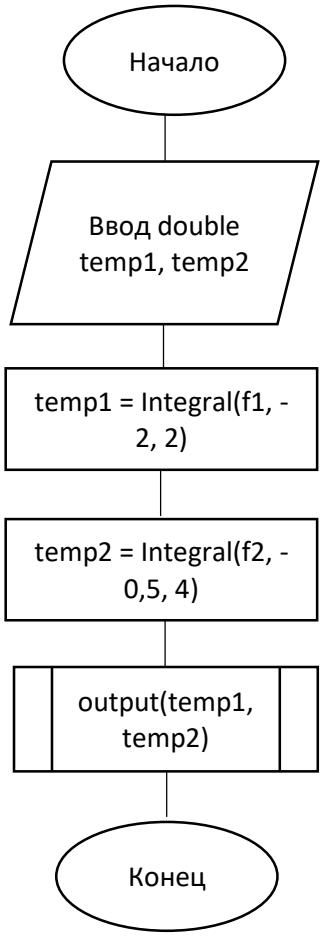
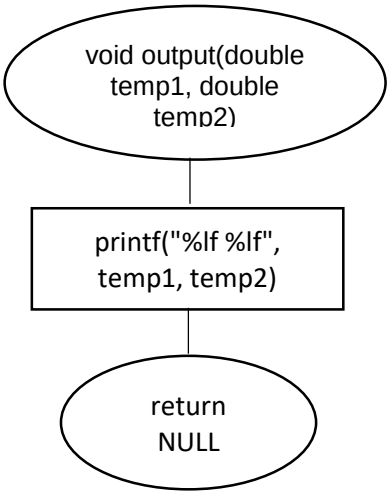


Схема функции output:



Текст программы:

```
#include <stdlib.h>

#include <math.h>

#define N 35 /*количество промежутков на которые делится график)*/

void output(double temp1, double temp2);

double Integral(double (*f) (double), double a, double b){/*функция
возвращает   и принимает число типа double */

double result = 0;

int i = 0;

double step = (b - a) / N;/* длина промежутка(сумма площадей промежутков)*/

for(i = 1; i < N; i++){

result += f(a + step * i);/*формула трапеции */

}

result += (f(a) + f(b)) / 2;

result *= step;

return result;

}

double f1(double x){/*вычисляем площадь помежутка*/

return sqrt(1 + x + x*x);

}

double f2(double x){/*вычисляем площадь помежутка*/

return x*sin(x);

}

void output(double temp1, double temp2){

printf("%lf %lf", temp1, temp2);

}

int main(){

double temp1 = Integral(f1, -2, 2);

double temp2 = Integral(f2, 0.5, 4);
```

```
output(temp1, temp2);
return 0;
}
```